Logika, vyjadrovacia sila a formálna verifikácia

Ján Mazák

FMFI UK Bratislava

Cieľom tejto prednášky je umožniť vám získať predstavu o matematickej logike a niektorých jej významných výsledkoch. Prednáška zároveň naznačí, prečo dotazovacie jazyky majú obmedzenú vyjadrovaciu silu. K tvrdeniam uvedeným v prezentácii pristupujte opatrne, podané sú v zjednodušenej forme a občas sú zamlčané technickejšie predpoklady.

Prednáška je informačná; obsah nebude na skúške.

Jazyk logiky 1. rádu

- Logické symboly: spojky $\neg, \wedge, \vee, \rightarrow$, kvantifikátory $\forall, \exists$ a rovnosť $=$.
- Indivíduové premenné $x, y, \ldots$ a pomocné symboly (zátvorky, čiarka).
- Mimologické symboly si volíme podľa toho, o čom chceme hovoriť:
  - predikátové symboly pre vzťahy, napr. binárny R;
  - funkčné symboly pre funkcie, napr. unárny f;
  - indivíduové konštanty pre konkrétne objekty, napr. c.

Počet argumentov symbolu je jeho *arita*.

Interpretácia určí konkrétne objekty, vzťahy a funkcie. Napr. nad $\mathbb{N}$ môžeme c interpretovať ako 0, f ako $n \mapsto n + 1$ a R ako $\leq$.

Teória: čiastočné usporiadanie

Teória je množina uzavretých formúl (bez voľných premenných) vo zvolenom jazyku. Napr. jazyk má jeden binárny predikátový symbol R a teória T obsahuje formuly

$$\forall x\ R(x,x) \qquad \text{reflexivita},$$
$$\forall x \forall y\ (R(x,y) \wedge R(y,x) \to x = y) \qquad \text{antisymetria},$$
$$\forall x \forall y \forall z\ (R(x,y) \wedge R(y,z) \to R(x,z)) \qquad \text{tranzitivita}.$$

To, či je zápis formulou, je čisto syntaktická otázka. Zatiaľ neurčujeme množinu prvkov ani konkrétny význam R.
O pravdivosti formuly hovoríme až po voľbe domény a interpretácie mimologických symbolov.

Štruktúra

Nech $\mathcal{L}$ je jazyk logiky prvého rádu. Štruktúrou pre jazyk $\mathcal{L}$ nazývame dvojicu $\mathcal{M} = (D, i)$, kde

- D je ľubovoľná neprázdna množina nazývaná *doména* štruktúry $\mathcal{M}$;
- i je zobrazenie, nazývané *interpretačná funkcia*, ktoré
  - každej indivíduovej konštante c jazyka $\mathcal{L}$ priraďuje prvok $i(c) \in D$;
  - každému predikátovému symbolu P s aritou n priraďuje reláciu $i(P) \subseteq D^n$;
  - každému funkčnému symbolu f s aritou n priraďuje funkciu $i(f)\colon D^n \to D$.

Rovnosť sa interpretuje ako rovnosť prvkov domény D.

Model teórie

Model teórie T je štruktúra, v ktorej sú splnené všetky formuly z T. Píšeme $\mathcal{M} \models T$.

- $D = \mathbb{N}$, $i(R) = \leq$: model našej teórie T.
- $D = \mathcal{P}(\{1, 2\})$, $i(R) = \subseteq$: tiež model T. ($\mathcal{P}(A)$ označuje množinu všetkých podmnožín A.)
- $D = \mathbb{Z}$, $i(R) = <$: nie je model T, neplatí reflexivita.

Rovnaká teória môže mať rôzne modely, konečné aj nekonečné.

Teórie pre konkrétne oblasti matematiky

Pri opise konkrétnej oblasti matematiky si vyberáme konkrétnu teóriu a jej formulám hovoríme axiómy.

Peanova aritmetika (PA): aritmetika prirodzených čísel. Napr. jedna z axióm:

$$\forall x\ (x + 0 = x).$$

ZFC: teória množín. Napr. axióma extenzionality:

$$\forall A \forall B\ \big(\forall x\,(x \in A \leftrightarrow x \in B) \rightarrow A = B\big).$$

Množiny s rovnakými prvkami sa rovnajú.

Pravdivosť v modeli a vyplývanie z teórie

Pre uzavretú formulu F zápis $\mathcal{M} \models F$ znamená, že F je splnená v štruktúre $\mathcal{M}$ („pravdivá pre konkrétnu interpretáciu mimologických symbolov“).

$T \models F$: formula F platí v každom modeli teórie T.

Pre našu teóriu čiastočného usporiadania uvažujme

$$F = \forall x \forall y \ (R(x, y) \vee R(y, x)).$$

- Pre $D = \mathbb{N}$ a $i(R) = \leq$ platí F.
- Pre $D = \mathcal{P}(\{1, 2\})$ a $i(R) = \subseteq$ neplatí F: množiny $\{1\}$ a $\{2\}$ sú neporovnateľné.

Preto $T \not\models F$ a $T \not\models \neg F$ — formula F je od teórie T nezávislá.

Formálny dôkaz a dokázateľnosť

Zvoľme odvodzovacie pravidlá dokazovacieho kalkulu. Dôkaz formuly F z teórie T je konečná postupnosť $F_1, \ldots, F_n = F$, kde každá formula F_i

- patrí do T, alebo
- vznikne z predchádzajúcich formúl použitím odvodzovacieho pravidla.

Napr. pravidlo modus ponens:

$$\frac{A \qquad A \to B}{B}.$$

$T \vdash F$: existuje takýto formálny dôkaz F z T.

Rezolvencia

Rezolvencia operuje nad klauzulami (disjunkciami literálov) a odvodzuje nové klauzuly, kým nenájde spor (prázdnu klauzulu).

$$\frac{(A \vee C_1 \vee C_2) \qquad (\neg A \vee B_1 \vee B_2)}{C_1 \vee C_2 \vee B_1 \vee B_2}$$

Dôkaz sporom: pri dôkaze F z T pridáme $\neg F$ a po prepise do klauzúl zachovávajúcom splniteľnosť odvodíme prázdnu klauzulu.
Korektnosť: rezolventa vyplýva z rodičovských klauzúl.

Používame tu malú sadu pravidiel na odvodenie prázdnej klauzuly, ale dôkaz zostáva postupnosťou algoritmicky ľahko overiteľných krokov, rovnako ako v mnohých ďalších dokazovacích systémoch, ktoré logici počas dejín vytvorili.

Rezolvencia: príklad dôkazu

Dokážeme nesplniteľnosť množiny

$$\{A \vee B, \qquad \neg A, \qquad \neg B\}.$$

1. Rezolvujeme $A \vee B$ s $\neg A$, dostaneme B.
2. Rezolvujeme B s $\neg B$, dostávame prázdnu klauzulu, tá je nesplniteľná.

Rezolvencia v logike 1. rádu: unifikácia

Čo možno usúdiť z nasledujúcich dvoch formúl?

$$\forall y\, P(f(y), y), \qquad \forall x\, (\neg P(x, d) \vee R(x))$$

Dosadenie popisujeme pomocou substitúcie:

$$\sigma = \{x \mapsto f(d),\ y \mapsto d\}.$$

Po substitúcii σ majú komplementárne literály rovnaké argumenty predikátu (preto σ nazývame *unifikátor*):
$P(f(d), d)$ a $\neg P(f(d), d)$.
Jedným z dôsledkov uvedených dvoch formúl je teda $R(f(d))$.

Pravdivosť a dokázateľnosť

Uvažujme jazyk logiky 1. rádu a v ňom teóriu T a formulu F.

Veta o korektnosti logiky 1. rádu

Ak $T \vdash F$, tak $T \models F$.

Veta o úplnosti logiky 1. rádu

Ak $T \models F$, tak $T \vdash F$.

Úplnosť nie je samozrejmá: $T \models F$ hovorí o všetkých modeloch, aj o tých s nespočítateľnou doménou, ktorej prvky nemožno algoritmicky enumerovať.
Pre platnú formulu musí existovať dôkaz. Pre neplatnú musí existovať protipríklad (model, v ktorom neplatí). Ani jedno z toho však nemusíme vedieť algoritmicky nájsť.

Nerozhodnuteľné problémy

- Rozhodnuteľný problém: existuje algoritmus, ktorý vždy skončí a odpovie správne áno alebo nie.
- Čiastočne rozhodnuteľný problém: algoritmus vždy potvrdí odpoveď áno, ak je správna; pri odpovedi nie môže bežať navždy.
- Ani čiastočne rozhodnuteľný: neexistuje ani takýto jednostranne zaručený algoritmus.

Príklady nerozhodnuteľných problémov:

- *Problém zastavenia programu:* zastaví sa daný program pre daný vstup?
- *Ekvivalencia dvoch programov v C:* majú pre každý vstup rovnaký výsledok?

Platnosť formuly vo FOL

Veta

Platnosť formuly v prvorádovej logike je nerozhodnuteľná.

Z korektnosti a úplnosti: formula je platná práve vtedy, keď má dôkaz. Systematicky enumerujeme všetky dôkazy; ak je formula platná, jej dôkaz skôr či neskôr nájdeme. Tak dostávame čiastočný rozhodovací postup.

Na hľadaní dôkazov sú založené automatické dokazovače. Rezolvencia s unifikáciou sa vo FOL v praxi osvedčila ako účinná metóda automatického dokazovania. Opakovane odvodzujú pomocou heuristík nové a nové dôsledky (v súlade s pravidlami) a dúfajú, že raz trafia kompletný dôkaz.

Otázka, či veta FOL *nie je* platná, nie je ani čiastočne rozhodnuteľná.

Splniteľnosť v prvorádovej logike

Veta

Splniteľnosť v prvorádovej logike je nerozhodnuteľná; nie je ani čiastočne rozhodnuteľná.

Kuriozita: monadická prvorádová logika (MFOL) nemá funkčné symboly (ani konštanty) a jej predikáty majú len jeden argument.
Platnosť formuly v MFOL je rozhodnuteľná. Ak pridáme čo len jeden predikát arity 2, platnosť formuly už nie je rozhodnuteľná.

Bezpečné databázové dotazy

Dotaz je doménovo nezávislý, ak rozšírenie domény o ďalšie hodnoty bez zmeny tabuliek nezmení jeho výsledok.

- $\neg\mathrm{Zamestnanec}(x)$ nie je doménovo nezávislý: vracia aj hodnoty mimo databázy. Pri rozšírení domény pribudnú ďalšie výsledky.
- $\mathrm{Osoba}(x) \wedge \neg\mathrm{Zamestnanec}(x)$ vyberá iba osoby uložené v tabuľke, ktoré nie sú zamestnancami. Rozšírenie domény výsledok nezmení.

Relačná algebra a relačné jadro SQL umožňujú iba *bezpečné dotazy*: výsledok vieme vypočítať z konečných dát, bez prehľadávania ľubovoľnej domény.
Pre FOL nemožno algoritmicky rozhodnúť, či je zadaný dotaz doménovo nezávislý, preto bezpečnosť zaručujeme obmedzením povolených zápisov (priamo syntaxou dotazovacieho jazyka).

Efektívna axiomatizovateľnosť

Teória je efektívne axiomatizovateľná, ak spĺňa tieto požiadavky na algoritmickú prácu:

- *Symboly:* vieme algoritmicky rozpoznať ich zápis.
- *Termy a formuly:* vieme rozhodnúť, či je zápis správny.
- *Axiómy:* algoritmus ich postupne všetky vypíše; môže ich byť nekonečne veľa.
- *Odvodzovacie pravidlá:* vieme algoritmicky overiť ich použitie.

Potom môžeme systematicky enumerovať všetky dôkazy a dokázateľné formuly.

Sémantická a negačná úplnosť

Doterajší pojem úplnosti, tzv. sémantická úplnosť formálneho systému, hovoril o tom, že pravdivé veci možno dokázať: Pre každú teóriu T a uzavretú formulu F, ak $T \models F$, tak $T \vdash F$.
Zaujímavá je však aj negačná úplnosť teórie T: pre každú uzavretú formulu F (v jazyku tejto teórie) platí $T \vdash F$ alebo $T \vdash \neg F$.

Prvá Gödelova veta o neúplnosti

Veta

Prvorádová teória T nemôže mať súčasne všetky štyri vlastnosti:

1. je konzistentná (nedokazuje spor);
2. je efektívne axiomatizovateľná;
3. obsahuje dostatočnú aritmetiku (napr. PA);
4. je negačne úplná: pre každú uzavretú formulu F jej jazyka platí $T \vdash F$ alebo $T \vdash \neg F$.

V princípe: zmysluplné teórie nikdy nehovoria o všetkom, vždy zostanú nejaké tvrdenia, ktoré z teórie nemožno ani dokázať, ani vyvrátiť. Matematiku ako celok nemožno plne axiomatizovať.

Druhá Gödelova veta o neúplnosti

Veta

Ak S je bezosporný efektívne axiomatizovateľný formálny systém obsahujúci aritmetiku (napr. PA), jeho bezospornosť (pri obvyklej formalizácii) nemožno dokázať v rámci S.

Čiže dôkaz treba spraviť neformálnou metaúvahou alebo v rámci iného „silnejšieho“ formálneho systému. Ako však dokážeme bezospornosť tohto druhého systému? ...

Hranice vyjadrovacej sily FOL

- Vo FOL kvantifikujeme cez prvky domény; predikáty (v databáze tabuľky) sú pevne dané.
- Chceli by sme sa opýtať: „Existuje v databáze iná tabuľka disjunktná s tabuľkou R?“ Pri pevnej schéme to vo FOL vieme rozpísať cez jednotlivé tabuľky. Chceli by sme však vyberať tabuľku premennou.
- Ak pridáme premenné pre predikáty a kvantifikujeme ich, dostaneme logiku 2. rádu. Pri plnej sémantike premenné prechádzajú všetky možné relácie, nielen existujúce tabuľky.
- Ani dosiahnuteľnosť v grafe sa nedá vyjadriť prvorádovými formulami, ako si čochvíľa dokážeme.

Ako vyjadriť tranzitívny uzáver

Majme binárnu reláciu reprezentovanú predikátom R. Pre každé $n \geq 1$ označme $R_n(a, b)$ formulu

$$\exists x_1 \, \exists x_2 \cdots \exists x_n \big(R(a, x_1) \wedge R(x_1, x_2) \wedge \cdots \wedge R(x_n, b) \big).$$

Ďalej nech $R_0 = R$ a nech

$$R^*(a, b) \quad \Leftrightarrow \quad \text{existuje } n \text{ také, že platí } R_n(a, b).$$

Naše R^* vyjadruje tranzitívny uzáver relácie R.

Dá sa však R^* vyjadriť prvorádovou formulou?

Kompaktnosť logiky 1. rádu

Veta o kompaktnosti

Každá nesplniteľná teória má konečnú podmnožinu, ktorá je nesplniteľná.

Ekvivalentne: ak má každá konečná podmnožina teórie T model, potom má model aj celá teória T.

- Teória T pritom môže byť nekonečná.
- Výsledný model nemusí byť konečný. Napr. požiadavky „aspoň n prvkov“ pre každé $n \geq 1$: každý konečný výber má konečný model, všetky spolu iba nekonečný.

Tranzitívny uzáver: dôkaz kompaktnosťou

Predpokladajme, že R^* sa dá vyjadriť prvorádovou formulou vo všetkých štruktúrach. Uvažujme teóriu

$$T = \{R^*(a, b)\} \cup \{\neg R_n(a, b) : n \geq 0\}.$$

1. T nemá model: ak $R^*(a, b)$, tak pre nejaké n platí $R_n(a, b)$.
2. Každá konečná podmnožina T má model: pre dostatočne veľké n vyhovuje cesta z a do b dĺžky $n + 2$.

Tieto dve pozorovania sú v spore s kompaktnosťou FOL.

Formulou logiky 1. rádu tranzitívny uzáver nevieme vyjadriť.

(Nevyjadriteľnosť sa dá dokázať aj pre konečné modely.)

Databázy: prečo potrebujeme rekurziu

- Relačné jadro SQL bez rekurzie zodpovedá FOL (množinový model, bezpečné dotazy).
- Na dosiahnuteľnosť v grafe teda potrebujeme rozšírenie jazyka: rekurzia počítajúca najmenší pevný bod (*least fixed point*, LFP) prekonáva túto hranicu FOL.
- Najmenší pevný bod vypočítame už známou naivnou alebo seminaivnou evaluáciou.

Databázy: agregácie a FOL

- `MIN`, `MAX`: pri dostupnom usporiadaní nepridávajú nič navyše oproti FOL.
- `SUM`: pridáme do logiky číselný term $\sum_{x:\varphi(x)} t(x)$: súčet hodnôt $t(x)$ pre všetky riadky spĺňajúce $\varphi(x)$. Počet sčítancov závisí od dát, zložitá sémantika.
- `COUNT`: osobitný prípad súčtu, kde $t(x) = 1$; s aritmetikou vyjadrí aj paritu, ktorú obyčajná FOL nevyjadrí.
- `AVG`: nič navyše oproti `SUM + COUNT`.

Ani s agregáciami nemožno vyjadriť tranzitívny uzáver.

Prečo nepoužiť C alebo Python nad tabuľkami?

- Bezpečné relačné jadro bez rekurzie a všeobecných funkcií sa nad konečnou databázou vždy vyhodnotí. Pre ľubovoľný program nevieme všeobecne rozhodnúť ani to, či skončí.
- C aj Python sa v DBMS používať dajú. Funkcia v „neobmedzenom“ jazyku použitá ako súčasť dotazu je však pre optimalizátor len čierna skrinka.
- Vyjadrovaciu silu pridávame tam, kde je užitočná či nutná: napr. rekurzia pre hierarchické dáta.
- Zložitejšie kontroly konzistencie, ktoré sa ťažko vyjadrujú v SQL, možno napr. v PostgreSQL napísať ako trigger functions v Pythone.

Sú dva stromy relačnej algebry ekvivalentné?

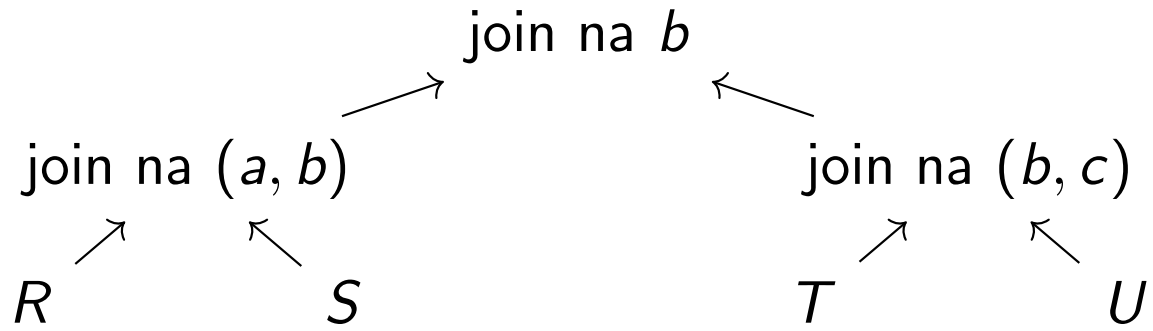
Optimalizácia je náročná už pre relačnú algebru: môžeme jeden strom nahradiť iným bez zmeny výsledku?

- Ekvivalentné stromy vrátia rovnaký výsledok na *každej konečnej databáze*.
- Pre plnú relačnú algebru (vrátane rozdielu) je ekvivalencia nerozhodnuteľná; nie je ani čiastočne rozhodnuteľná.

Optimalizátor používa dokázané pravidlá prepisovania; nemusí rozpoznať všetky ekvivalentné dvojice stromov.

Optimalizácia pri pevnom strome

Jednoduchšia úloha: zafixujeme strom v relačnej algebre, všetky joiny rátame ako merge join, vyberáme si iba poradie atribútov, podľa ktorého sa relácie pri výpočte joinu usporiadajú (zhoda sa posudzuje na celej n-tici spoločných atribútov):



- Naľavo (b, a) zaručí zoradenie podľa b aj pre horný join; (a, b) túto záruku nedáva.
- Napravo analogicky: (b, c), nie (c, b). Existujúci index môže zvýhodniť poradie nevhodné pre horný join.

Aj výber zoradení pre pevný strom merge joinov je vo všeobecnosti NP-ťažký. Úloha je však rozhodnuteľná.

Logika 2. rádu

Syntax a sémantiku logiky 1. rádu možno rozšíriť o premenné pre predikáty, ktoré je dovolené kvantifikovať. Dostaneme tak logiku 2. rádu (SO).

Ak sa pri kvantifikovaní predikátov obmedzíme na predikáty arity 1, dostaneme monadickú logiku 2. rádu (MSO). Takéto predikáty de facto zodpovedajú množinám prvkov domény, čiže v MSO možno vyjadriť bežné veci z teórie grafov apod.

Pri plnej sémantike: Použitie formúl 2. rádu zvyšuje vyjadrovaciu silu jazyka, ale oproti FOL stratíme kompaktnosť a úplnosť.

Deskriptívna zložitosť

Pri klasickom pohľade na zložitosť algoritmov skúmame počet krokov v závislosti od veľkosti vstupu.
Alternatívny pohľad: akú zložitú logickú formulu potrebujeme na popis daného problému?
Príklady rôzne zložitých formúl:

- existential SOL: $\exists X_1 \cdots \exists X_n \ F$, kde F je formula FOL
- universal SOL: $\forall X_1 \cdots \forall X_n \ F$, kde F je formula FOL

Formula je pevná, vstupom sú dáta (napr. graf).

Deskriptívna zložitosť

Existencia trojuholníka v grafe sa dá vyjadriť vo FOL:

$$\exists x\, \exists y\, \exists z\ \big(V(x) \wedge V(y) \wedge V(z) \wedge \\ \wedge\, x \neq y \wedge y \neq z \wedge z \neq x \wedge E(x,y) \wedge E(y,z) \wedge E(z,x)\big)$$

Predikát V popisuje vrcholy, E hrany. Formulu vyhodnocujeme na konkrétnom grafe, neoverujeme jej všeobecnú platnosť. Pre k kvantifikátorov vo formule preveríme $O(|V|^k)$ možností. Formula je pevná, rastie graf. Preto tento problém patrí do triedy P.

Deskriptívna zložitosť

Existencia 3-farbenia grafu: NP-úplný;
nevyjadriteľné vo FOL, vyjadriteľné v existential SOL:

$$\exists R\, \exists G\, \exists B \big(\forall x\, \forall y\ (E(x,y) \to \neg R(x) \vee \neg R(y)) \wedge \cdots \big)$$

(R, G, B sú predikáty vyjadrujúce jednotlivé farby).
Za doménu zoberieme vrcholy grafu. Množiny farieb nedeterministicky uhádneme; vyhodnotenie pevnej prvorádovej časti už trvá polynomiálny čas. Preto tento problém patrí do triedy NP.

Deskriptívna zložitosť

Konečné usporiadané vstupy; formula je pevná, rastú dáta.

- FOL s aritmetikou na pozíciách: trieda zložitosti AC^0 (vlastná podmnožina LOGSPACE)
- FOL + tranzitívny uzáver: non-deterministic LOGSPACE
- FOL + least fixed point operator: P
- existential SOL: NP
- universal SOL: co-NP
- SOL: PH (obsahuje NP aj coNP)
- SOL + least fixed point operator: EXPTIME

V poslednom riadku ide o pevný bod vyššieho rádu.

Lean: jazyk a interaktívny dokazovací systém

Lean je dôkazový asistent založený na teórii typov.

- Na dokázanie formuly treba skonštruovať term zodpovedajúceho typu.
- Lean vie verifikovať existujúce dôkazy (termy).
- Lean tiež ponúka tzv. „taktiky“, ktoré pomáhajú automatizovať konštrukciu dôkazu (napr. mu možno dať pokyn „sprav rozbor prípadov“, „nejako skús použiť túto lemu“, „aplikuj pravidlá násobenia v poli“).
- `mathlib` je rastúca knižnica formálne verifikovaných matematických tvrdení.

Lean: dôkaz priamou konštrukciou termu

A, B, C sú tvrdenia. Dôkaz tvrdenia A je v Leane hodnota typu A.

```
-- Direct proof term
theorem my_term (A B C : Prop) :
    (A ∧ C) → (B → A) :=
  fun h_ac : A ∧ C =>
    fun h_b : B =>
      h_ac.left
```

Funkcia dostane dôkaz $A \wedge C$, potom dôkaz B a vráti dôkaz A. Tým dokazuje celú implikáciu.

Lean: dôkaz pomocou taktík

Ten istý dôkazový term môžeme zostrojiť pomocou taktík:

```
theorem my_tactic_proof (A B C : Prop) :
    (A ∧ C) → (B → A) := by
  intro h_ac
  obtain ⟨h_a, h_c⟩ := h_ac
  intro h_b
  exact h_a
```

Výroky ako typy, dôkazy ako termy: Curryho-Howardova korešpondencia.

Lean: kód s dôkazom správnosti

Lean je aj funkcionálny programovací jazyk.

Úloha: napísať funkciu pre maximum z neprázdneho zoznamu a dokázať, že výsledok patrí do zoznamu a je najväčší.

Možnosť 1: návratový typ obsahuje výsledok aj dôkaz správnosti.

```
def find_max : (L : List ℕ) → L ≠ [] →
        { x // x ∈ L ∧ ∀ y ∈ L, y ≤ x }
  | [], h => by contradiction
  | [a], _ => ⟨a, by sorry⟩
  | a :: b :: tail, _ =>
      let ⟨m, ⟨hm_mem, hm_max⟩⟩ := find_max (b :: tail) (by simp)

      if h : m ≤ a then ⟨a, by sorry⟩
      else ⟨m, by sorry⟩
```

Dôkazové časti sú vynechané (`sorry`).

Lean: samostatný dôkaz správnosti

Možnosť 2: funkcia vracia bežnú hodnotu a jej správnosť vyjadríme samostatnou vetou.
Veta požaduje, aby výsledok patril do zoznamu a bol najväčší.

```
def find_max : (L : List ℕ) → L ≠ [] → ℕ := sorry

theorem find_max_correct : (L : List ℕ) → (h : L ≠ []) →
  find_max L h ∈ L ∧ ∀ y ∈ L, y ≤ find_max L h := by sorry
```

Náčrt deklarácií; implementácia a dôkaz sú vynechané.

Lean: dokázateľne správny kód

Čo presne znamená rozdeliť zoznam na párne a nepárne prvky?
Zachytávajú tieto podmienky všetko, čo od programu chceme?
Ukážka špecifikácie; dôkazy sú vynechané.

```
theorem split_parity_length (xs : List Nat) :
  (split_parity xs).1.length + (split_parity xs).2.length = xs.length :=
     by sorry

theorem split_parity_all_even (xs : List Nat) :
  ∀ x ∈ (split_parity xs).1, Even x := by sorry

theorem split_parity_all_odd (xs : List Nat) :
  ∀ x ∈ (split_parity xs).2, Odd x := by sorry

theorem split_parity_mem (xs : List Nat) (x : Nat) :
  x ∈ xs ↔ x ∈ (split_parity xs).1 ∨ x ∈ (split_parity xs).2 := by
    sorry
```

Dôkaz je len taký dobrý ako špecifikácia

Chýba zachovanie násobností. Požiadavky na dĺžku, paritu a členstvo dovoľujú

$$[2, 4, 4] \longmapsto ([2, 2, 4], []).$$

- Súčet dĺžok výstupných zoznamov je tri.
- Prvý výstup obsahuje iba párne čísla a druhý iba nepárne.
- Vstup a výstup obsahujú tie isté rôzne hodnoty, ale nie tie isté násobnosti.

Dôkaz je len taký dobrý ako špecifikácia

Chýba stabilita: aj keď zachováme násobnosti, slabá špecifikácia povoľuje meniť poradie.
Popri podmienkach pre paritu a dĺžku požadujeme:

```
theorem split_parity_stable (xs : List Nat) :
  (split_parity xs).1.Sublist xs ∧
  (split_parity xs).2.Sublist xs := by sorry
```

Podzoznam vznikne vynechaním prvkov bez zmeny poradia. Spolu s paritou a súčtom dĺžok tým zaručíme aj správne násobnosti.

Stabilita je veľmi dôležitá aj pri testovaní: pri nedeterministickom výstupe je porovnanie s jediným očakávaným zoznamom nespoľahlivé.
V praxi môže premiešanie časovo zoradených záznamov pokaziť napr. výpočet zmien medzi susednými meraniami.

Logické solvery v praxi

- Logické solvery sa stali jednými z najrýchlejších nástrojov na mnohé praktické aj matematické problémy.
- Základ algoritmov pre SAT poznáme od 60. rokov; praktický prelom prišiel okolo 2000. CDCL (Conflict-Driven Clause Learning) sa pri backtrackingu učí z konfliktov nové klauzuly a v prehľadávanom priestore odrezáva celé podstromy.
- Amazon používa Z3 od Microsoftu pri kontrole prístupu k S3 (cloudové úložisko). Ide o SMT solver, ktorého princíp si o chvíľu vysvetlíme.
- Výsledkom môže byť overiteľný certifikát: ohodnotenie (SAT) alebo dôkaz nesplniteľnosti (UNSAT).
- Certifikáty využijeme v Leane pri overovaní kódu a výpočtov, v priemysle aj matematickom výskume.

Problém SAT

- Má boolovská formula ohodnotenie, v ktorom je splnená?
- Pre danú boolovskú formulu hľadáme priradenie hodnôt pravda/nepravda jej premenným, pri ktorom je formula pravdivá. Existuje také priradenie?
- Vstupom býva formula v CNF: konjunkcia klauzúl, ktorú už poznáme z rezolvencie.
- SAT solver nájde vyhovujúce priradenie alebo rozhodne, že žiadne neexistuje.
- *Predspracovanie:* rezolvenčná eliminácia znižuje počet premenných aj za cenu väčších klauzúl, čo môže urýchliť následné hľadanie.
- Hoci je SAT NP-úplný problém, solvery sú prekvapujúco rýchle, neraz aj pre tisíce či milióny premenných.

Praktické využitie SAT solverov

- Verifikácia hardvéru: existuje vstup, na ktorom sa obvod pred optimalizáciou a po nej správa odlišne?
- Softvérové balíky: existuje kompatibilný výber verzií?
- Existuje farbenie grafu najviac k farbami?
- Existuje v grafe hamiltonovská kružnica?

Každý problém najprv preložíme na boolovskú formulu. Do SAT možno zakódovať prakticky ľubovoľnú konečnú kombinatorickú úlohu; veľkosť prekladu však nemusí byť polynomiálna (pri bežných kombinatorických úlohách však je).

Čo je SMT solver?

SMT (Satisfiability Modulo Theories) kombinuje logické spojky s podmienkami zo zvolených teórií, napr. aritmetiky.
Pre $x, y \in \mathbb{Z}$ hľadáme riešenie:

$$(x = 0 \vee y = 0) \wedge x + y = 5 \wedge x > y.$$

- Podporované teórie: lineárna aritmetika nad celými a reálnymi číslami, funkcie bez určenej interpretácie, polia, bitové vektory.
- Úplné rozhodovanie sa týka vybraných teórií a fragmentov, nie ľubovoľnej matematiky.

Ako pracuje SMT solver?

- Základom je SAT solver, ale atómami môžu byť aj matematické tvrdenia, napr. $x + y = 5$ alebo $x > y$. SAT časť ich význam nepozná; berie ich ako boolovské premenné.
- SAT solver navrhuje, ktoré atómy majú byť pravdivé a ktoré nepravdivé, aby vyhovel logickým spojkám.
- Špecializované moduly pre jednotlivé teórie priebežne kontrolujú, či sú tieto voľby zlučiteľné s doterajšími podmienkami v príslušnej teórii.
- Napr. modul pre lineárnu aritmetiku odhalí spor v nerovnostiach a oznámi ho SAT solveru. Ten sa z konfliktu poučí a hľadá iné ohodnotenie.

Dokázali sme zamýšľané tvrdenie?

Pri počítačovom dokazovaní aj databázových dotazoch je kľúčová formalizácia zadania z prirodzeného jazyka. Môže ju vytvoriť človek alebo LLM.

- Algoritmické strojové overenie či výpočet nezaručí, že zápis vystihuje pôvodné zadanie.
- „Nech n je prvý krok, v ktorom sa program P zastaví.“ Tento zápis predpokladá, že taký krok existuje.
- Overiť tento skrytý predpoklad znamená vyriešiť problém zastavenia.

Už overenie vernosti formalizácie môže byť nerozhodnuteľné, dokonca ťažšie než problém zastavenia. V princípe k ľubovoľne ťažkému výpočtovému problému tak existuje aspoň tak ťažký formalizačný problém. Ani Lean, ani LLM nás od toho nezachránia.

Bastounis, Circelli, Hansen (2026), Navier-Stokes lost in translation.